

Perfekt — ich habe die kritischen aktuellen Details gefunden:

#### 🔍 Wichtige Findings:

- **3** ``update-electron-app`` ist nur für **\*\*gepackte\*\*** Apps gedacht (nicht in der Entwicklung) — ``electron-is-dev`` als Check empfohlen
- **2** `electron-updater` unterstützt mehrere Provider out-of-the-box (GitHub Releases, S3, generischer HTTP-Server), nur 2 Zeilen Code nötig
- **3** Update-Feed-URL-Pattern:  
``${server}/OWNER/REPO/${process.platform}-${process.arch}/${app.getVersion()}``
- **16** `llama.cpp` Streaming-Format: Standard SSE mit ``data: {json}`` Zeilen, terminiert mit ``stop:true`` und ``timings``-Objekt
- **13** Neu: `llama.cpp` unterstützt jetzt auch die Anthropic Messages API mit eigenem Streaming-Format unter ``/v1/messages``
- Bekannter Bug (Sept 2025): SSE-Fehlermeldungen bei Context-Overflow sind nicht RFC-konform — daher robustes Error-Handling im Client wichtig

## 🚀 Hermes Control Center — Stufe 5: Auto-Update + Download + Streaming-Chat

### Architektur-Überblick

```
{ } text
```



#### DIESE FINAL-STUFE

- Auto-Updater → `electron-updater` + GitHub Releases
  - └ Nur in gepackten Apps aktiv
  - └ Download-Progress an Renderer
  - └ User-Choice: "Jetzt installieren" oder "Später"
- In-App Download → Node `https` + `fs.createWriteStream`
  - └ Echter Fortschrittsbalken (Bytes/Total)
  - └ Geschwindigkeit (MB/s)
  - └ ETA-Berechnung
  - └ Pause/Resume/Cancel
  - └ Auto-Save in user-gewählten Ordner
- Chat-Interface → SSE-Streaming gegen `/v1/chat/completions`
  - └ Token-für-Token Anzeige

- └ Conversation-History
- └ System-Prompt konfigurierbar
- └ Stop-Button (AbortController)
- └ Markdown-Rendering (minimal, inline)
- └ Token-Counter + ms/Token Stats

## 1 main.js — Auto-Updater + In-App Downloader

{ } JavaScript



```
// =====  
// ERGÄNZUNG zu main.js – Auto-Updater + Downloader  
// =====  
  
const { autoUpdater } = require('electron-updater')  
const https = require('https')  
const fs = require('fs')  
const url = require('url')  
  
// — State für aktive Downloads —  
const activeDownloads = new Map() // downloadId → { req, stream, abort }  
  
// =====  
// AUTO-UPDATER – nur in gepackter App  
// =====  
  
function initAutoUpdater() {  
  // KRITISCH: Nur in gepackter App ausführen  
  if (!app.isPackaged) {  
    console.log('[Updater] Dev-Mode – Auto-Updater deaktiviert')  
    return  
  }  
  
  // Logging für Diagnose  
  autoUpdater.logger = {  
    info: (m) => pushUpdaterLog('info', m),  
    warn: (m) => pushUpdaterLog('warn', m),  
    error: (m) => pushUpdaterLog('error', m),  
    debug: (m) => pushUpdaterLog('debug', m)  
  }  
  
  // Auto-Download AUS – User entscheidet  
  autoUpdater.autoDownload = false  
  autoUpdater.autoInstallOnAppQuit = true  
  
  // — Events an Renderer weiterleiten —  
  autoUpdater.on('checking-for-update', () => {  
    sendUpdate('checking', { msg: 'Suche nach Updates...' })  
  })  
}
```

```

autoUpdater.on('update-available', (info) => {
  sendUpdate('available', {
    version: info.version,
    releaseDate: info.releaseDate,
    releaseNotes: typeof info.releaseNotes === 'string'
      ? info.releaseNotes
      : (info.releaseNotes || []).map(n => n.note).join('\n'),
    currentVersion: app.getVersion()
  })

  showNotification(
    '📦 Update verfügbar',
    `Version ${info.version} ist verfügbar. Öffne die App um zu aktualisieren.`
  )
})

autoUpdater.on('update-not-available', (info) => {
  sendUpdate('not-available', {
    version: info.version,
    currentVersion: app.getVersion()
  })
})

autoUpdater.on('download-progress', (progress) => {
  sendUpdate('progress', {
    percent: Math.round(progress.percent),
    bytesPerSec: progress.bytesPerSecond,
    transferred: progress.transferred,
    total: progress.total
  })
})

autoUpdater.on('update-downloaded', (info) => {
  sendUpdate('downloaded', {
    version: info.version
  })

  showNotification(
    '✅ Update bereit',
    `Version ${info.version} wurde heruntergeladen. Klicke auf "Installieren" in der App.`,
    true
  )
})

autoUpdater.on('error', (err) => {
  sendUpdate('error', { message: err.message })
  pushUpdaterLog('error', `Updater-Fehler: ${err.message}`)
})

// — Beim Start checken (verzögert) —————
setTimeout(() => {
  autoUpdater.checkForUpdates().catch(err =>
    console.error('[Updater] Initial check failed:', err)
  )
}, 5000)

// — Periodischer Check alle 4 Stunden —————

```

```

setInterval(() => {
  if (app.isPackaged) {
    autoUpdater.checkForUpdates().catch(() => {})
  }
}, 4 * 60 * 60 * 1000)
}

function sendUpdate(event, data) {
  mainWindow?.webContents.send('updater-event', { event, data })
}

function pushUpdaterLog(level, msg) {
  const entry = { ts: Date.now(), level, msg: `[Updater] ${msg}` }
  logBuffer.push(entry)
  if (logBuffer.length > 5000) logBuffer.shift()
  mainWindow?.webContents.send('log-line', { level, msg: entry.msg })
}

// =====
// IPC: AUTO-UPDATER
// =====

ipcMain.handle('updater-check', async () => {
  if (!app.isPackaged) {
    return {
      success: false,
      error: 'Updates funktionieren nur in der gepackten App.'
    }
  }
  try {
    const result = await autoUpdater.checkForUpdates()
    return { success: true, info: result?.updateInfo }
  } catch (err) {
    return { success: false, error: err.message }
  }
})

ipcMain.handle('updater-download', async () => {
  if (!app.isPackaged) {
    return { success: false, error: 'Nur in gepackter App verfügbar.' }
  }
  try {
    await autoUpdater.downloadUpdate()
    return { success: true }
  } catch (err) {
    return { success: false, error: err.message }
  }
})

ipcMain.handle('updater-install', () => {
  if (!app.isPackaged) return { success: false }
  // Beendet die App und installiert das Update
  autoUpdater.quitAndInstall(false, true)
  return { success: true }
})

ipcMain.handle('updater-get-version', () => ({
  version: app.getVersion(),

```

```

electron:    process.versions.electron,
node:        process.versions.node,
chrome:      process.versions.chrome,
isPackaged:  app.isPackaged
}))

// =====
//  IN-APP MODEL DOWNLOADER
//  → Echter Stream-Download mit Progress
// =====

ipcMain.handle('download-pick-folder', async () => {
  const result = await dialog.showOpenDialog(mainWindow, {
    title:      'Zielordner für Modell-Download wählen',
    properties: ['openDirectory', 'createDirectory'],
    defaultPath: store.get('downloadDir', app.getPath('downloads'))
  })

  if (!result.canceled && result.filePaths[0]) {
    store.set('downloadDir', result.filePaths[0])
    return { success: true, path: result.filePaths[0] }
  }
  return { success: false }
})

ipcMain.handle('download-start', async (_, options) => {
  const { downloadUrl, filename, targetDir } = options
  const downloadId = `dl_${Date.now()}_${Math.random().toString(36).slice(2,8)}`

  const finalDir = targetDir || store.get('downloadDir', app.getPath('downloads'))
  const finalPath = path.join(finalDir, filename)

  // Sicherstellen, dass der Ordner existiert
  try {
    fs.mkdirSync(finalDir, { recursive: true })
  } catch (e) {
    return { success: false, error: `Ordner kann nicht erstellt werden: ${e.message}` }
  }

  // Prüfen ob Datei schon existiert
  if (fs.existsSync(finalPath)) {
    return {
      success: false,
      error: 'Datei existiert bereits.',
      existsPath: finalPath
    }
  }

  // Download starten (asynchron, IPC-Events streamen Progress)
  startDownload(downloadId, downloadUrl, finalPath)

  return { success: true, downloadId, targetPath: finalPath }
})

function startDownload(downloadId, downloadUrl, targetPath, redirectsLeft = 5) {
  const startTs = Date.now()
  let totalBytes = 0

```

```

let downloaded = 0
let lastEmit = Date.now()
let lastBytes = 0
let cancelled = false

const cleanup = () => activeDownloads.delete(downloadId)

const parsedUrl = url.parse(downloadUrl)
const tempPath = targetPath + '.part'

const fileStream = fs.createWriteStream(tempPath)

const req = https.get({
  hostname: parsedUrl.hostname,
  path: parsedUrl.path,
  headers: {
    'User-Agent': 'HermesControlCenter/2.0',
    'Accept': '*/*'
  },
  timeout: 30000
}, (res) => {

  // — Redirect (HuggingFace nutzt 302) —————
  if (res.statusCode >= 300 && res.statusCode < 400 && res.headers.location) {
    fileStream.close()
    try { fs.unlinkSync(tempPath) } catch {}

    if (redirectsLeft <= 0) {
      sendDownloadEvent(downloadId, 'error',
        { message: 'Zu viele Redirects' })
      cleanup()
      return
    }

    // Redirect verfolgen
    const nextUrl = res.headers.location.startsWith('http')
      ? res.headers.location
      : `https://${parsedUrl.hostname}${res.headers.location}`

    startDownload(downloadId, nextUrl, targetPath, redirectsLeft - 1)
    return
  }

  // — Fehler —————
  if (res.statusCode !== 200) {
    fileStream.close()
    try { fs.unlinkSync(tempPath) } catch {}
    sendDownloadEvent(downloadId, 'error',
      { message: `HTTP ${res.statusCode}` })
    cleanup()
    return
  }

  totalBytes = parseInt(res.headers['content-length'] || '0', 10)

  sendDownloadEvent(downloadId, 'started', {
    filename: path.basename(targetPath),

```

```

    totalBytes,
    targetPath
  })

  // — Stream auf Datei schreiben —————
  res.on('data', (chunk) => {
    if (cancelled) return
    downloaded += chunk.length

    // Throttled Progress-Events (max 10x/s)
    const now = Date.now()
    if (now - lastEmit >= 100) {
      const elapsed = (now - startTs) / 1000
      const speedBps = downloaded / Math.max(elapsed, 0.1)
      const remaining = totalBytes > 0
        ? (totalBytes - downloaded) / Math.max(speedBps, 1)
        : 0
      const percent = totalBytes > 0
        ? Math.round((downloaded / totalBytes) * 100)
        : 0

      sendDownloadEvent(downloadId, 'progress', {
        downloaded,
        totalBytes,
        percent,
        speedBps: Math.round(speedBps),
        etaSeconds: Math.round(remaining)
      })

      lastEmit = now
      lastBytes = downloaded
    }
  })

  res.pipe(fileStream)

  fileStream.on('finish', () => {
    fileStream.close(() => {
      if (cancelled) return

      // Temp-Datei umbenennen
      try {
        fs.renameSync(tempPath, targetPath)
      } catch (err) {
        sendDownloadEvent(downloadId, 'error', {
          message: `Umbenennen fehlgeschlagen: ${err.message}`
        })
        cleanup()
        return
      }

      const elapsed = (Date.now() - startTs) / 1000

      sendDownloadEvent(downloadId, 'completed', {
        targetPath,
        totalBytes: downloaded,
        elapsedSeconds: Math.round(elapsed),
        avgSpeedBps: Math.round(downloaded / Math.max(elapsed, 1))
      })
    })
  })
}

```

```

    })

    showNotification(
      '✅ Download abgeschlossen',
      `${path.basename(targetPath)} wurde gespeichert.`
    )

    cleanup()
  })
})

res.on('error', (err) => {
  fileStream.close()
  try { fs.unlinkSync(tempPath) } catch {}
  sendDownloadEvent(downloadId, 'error', { message: err.message })
  cleanup()
})
})

req.on('error', (err) => {
  fileStream.close()
  try { fs.unlinkSync(tempPath) } catch {}
  sendDownloadEvent(downloadId, 'error', { message: err.message })
  cleanup()
})

req.on('timeout', () => {
  req.destroy()
  fileStream.close()
  try { fs.unlinkSync(tempPath) } catch {}
  sendDownloadEvent(downloadId, 'error', { message: 'Timeout' })
  cleanup()
})

// — In activeDownloads-Map speichern für Cancel —————
activeDownloads.set(downloadId, {
  cancel: () => {
    cancelled = true
    req.destroy()
    fileStream.close()
    try { fs.unlinkSync(tempPath) } catch {}
    sendDownloadEvent(downloadId, 'cancelled', {})
    cleanup()
  }
})
}

function sendDownloadEvent(downloadId, event, data) {
  mainWindow?.webContents.send('download-event', { downloadId, event, data })
}

ipcMain.handle('download-cancel', (_, downloadId) => {
  const dl = activeDownloads.get(downloadId)
  if (dl) {
    dl.cancel()
    return { success: true }
  }
})

```



```

    return { success: false, error: 'Download nicht gefunden' }
  })

ipcMain.handle('download-list', () => {
  return Array.from(activeDownloads.keys())
})

// =====
// INIT-AUFRUF nach createWindow()
// =====

// In whenReady() ergänzen:
//   createWindow()
//   createTray()
//   initAutoUpdater() ← NEU

```

## 2 preload.js — Erweiterung

{ } JavaScript



```

// — Auto-Updater —
updaterCheck:      () => ipcRenderer.invoke('updater-check'),
updaterDownload:   () => ipcRenderer.invoke('updater-download'),
updaterInstall:    () => ipcRenderer.invoke('updater-install'),
updaterGetVersion: () => ipcRenderer.invoke('updater-get-version'),
onUpdaterEvent:    (cb) => ipcRenderer.on('updater-event',
  (_, d) => cb(d)),

// — Downloader —
downloadPickFolder: () => ipcRenderer.invoke('download-pick-folder'),
downloadStart:      (o) => ipcRenderer.invoke('download-start', o),
downloadCancel:     (id) => ipcRenderer.invoke('download-cancel', id),
downloadList:       () => ipcRenderer.invoke('download-list'),
onDownloadEvent:    (cb) => ipcRenderer.on('download-event',
  (_, d) => cb(d)),

```

## 3 renderer/pages/chat.js — Streaming-Chat-Interface

{ } JavaScript



```

// =====
// CHAT-INTERFACE mit SSE-Streaming
// Endpoint: /v1/chat/completions mit stream:true
// =====

```

```

const chatModule = (() => {

  // — State —————
  const state = {
    messages:      [],           // {role, content, ts, tokens?, elapsed?}
    systemPrompt:   'Du bist Hermes, ein hilfreicher KI-Assistent.',
    temperature:    0.7,
    maxTokens:      2048,
    streaming:      false,
    abortController: null,
    currentStream:   null,
    port:           8080
  }

  // — Seite rendern —————
  function renderChatPage(container) {
    container.innerHTML = `
      <div class="chat-layout">

        <!-- — HEADER ————— -->
        <div class="chat-header">
          <div class="chat-title">
            <h1>👉 Chat</h1>
            <span class="chat-endpoint" id="chat-endpoint">—</span>
          </div>
          <div class="chat-actions">
            <button class="btn btn-pick btn-sm" id="chat-settings-btn">
              ⚙️ Optionen
            </button>
            <button class="btn btn-sm" id="chat-clear-btn">
              🗑 Verlauf löschen
            </button>
          </div>
        </div>

        <!-- — OPTIONS PANEL (collapsible) ————— -->
        <details class="chat-options" id="chat-options">
          <summary>Erweiterte Optionen</summary>
          <div class="chat-options-grid">
            <div class="param-field">
              <label>System-Prompt</label>
              <textarea id="chat-system" rows="2">${escapeHtml(state.systemPrompt)}</textarea>
            </div>
            <div class="chat-options-row">
              <div class="param-field">
                <label>Temperature (${state.temperature})</label>
                <input id="chat-temp" type="range"
                  min="0" max="2" step="0.1"
                  value="${state.temperature}">
              </div>
              <div class="param-field">
                <label>Max Tokens</label>
                <input id="chat-maxtok" type="number"
                  value="${state.maxTokens}">
              </div>
            </div>
          </div>
        </details>
      `
  }
}

```

```

    </div>
</details>

<!-- — MESSAGES ————— -->
<div id="chat-messages" class="chat-messages">
  <div class="chat-empty">
    <div class="ce-icon">💡</div>
    <p>Starte das Gespräch.</p>
    <small>Tipp: Stelle eine Frage auf Deutsch oder Englisch.</small>
    <div class="chat-suggestions">
      <button class="chat-suggestion" data-q="Erkläre Quantencomputer in 3 Sätzen.">
        💡 Quantencomputer erklären
      </button>
      <button class="chat-suggestion" data-q="Schreibe eine Python-Funktion, die prüft ob eine Zahl
prim ist.">
        🐍 Python-Code
      </button>
      <button class="chat-suggestion" data-q="Was ist der Unterschied zwischen REST und GraphQL?">
        🌐 REST vs GraphQL
      </button>
    </div>
  </div>
</div>

<!-- — INPUT BAR ————— -->
<div class="chat-input-bar">
  <textarea id="chat-input" rows="2"
    placeholder="Nachricht eingeben... (Strg+Enter zum Senden)"
    class="chat-input"></textarea>
  <div class="chat-input-actions">
    <button class="btn btn-start" id="chat-send-btn" disabled>
      ▶ Senden
    </button>
    <button class="btn btn-stop hidden" id="chat-stop-btn">
      ■ Stopp
    </button>
  </div>
</div>

<!-- — STATUS BAR ————— -->
<div class="chat-statusbar">
  <span id="chat-status">Bereit</span>
  <span id="chat-stats"></span>
</div>
</div>
,

attachChatListeners()
updateEndpoint()
checkServerStatus()
}

// — Listener —————
function attachChatListeners() {
  const input    = document.getElementById('chat-input')
  const sendBtn  = document.getElementById('chat-send-btn')
  const stopBtn  = document.getElementById('chat-stop-btn')

```

```

const clearBtn = document.getElementById('chat-clear-btn')
const tempEl = document.getElementById('chat-temp')
const maxEl = document.getElementById('chat-maxtok')
const sysEl = document.getElementById('chat-system')

if (input) {
  input.addEventListener('input', () => {
    if (sendBtn) sendBtn.disabled = !input.value.trim() || state.streaming
  })
  input.addEventListener('keydown', (e) => {
    if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) {
      e.preventDefault()
      if (!state.streaming) sendMessage()
    }
  })
}

sendBtn?.addEventListener('click', sendMessage)
stopBtn?.addEventListener('click', stopStreaming)

clearBtn?.addEventListener('click', () => {
  if (state.messages.length === 0) return
  if (!confirm('Chat-Verlauf wirklich löschen?')) return
  state.messages = []
  renderMessages()
})

tempEl?.addEventListener('input', (e) => {
  state.temperature = parseFloat(e.target.value)
  const label = tempEl.previousElementSibling
  if (label) label.textContent = `Temperature (${state.temperature})`
})

maxEl?.addEventListener('change', (e) => {
  state.maxTokens = parseInt(e.target.value) || 2048
})

sysEl?.addEventListener('change', (e) => {
  state.systemPrompt = e.target.value
})

// Vorschlags-Buttons
document.querySelectorAll('.chat-suggestion').forEach(btn => {
  btn.addEventListener('click', () => {
    if (input) {
      input.value = btn.dataset.q
      input.dispatchEvent(new Event('input'))
      input.focus()
    }
  })
})

function updateEndpoint() {
  const ep = document.getElementById('chat-endpoint')
  if (ep) {
    ep.textContent = `🌐 http://127.0.0.1:${state.port}/v1`
  }
}

```

```

    }
}

async function checkServerStatus() {
    const result = await window.electronAPI.apiHealthCheck(state.port)
    const status = document.getElementById('chat-status')
    const input = document.getElementById('chat-input')
    const sendBtn = document.getElementById('chat-send-btn')

    if (result.ok) {
        if (status) status.innerHTML =
            '<span class="dot dot-on"></span> Server bereit'
        if (input) input.disabled = false
    } else {
        if (status) status.innerHTML =
            '<span class="dot dot-off"></span> Server nicht erreichbar'
        if (input) input.disabled = true
        if (sendBtn) sendBtn.disabled = true
    }
}

// — Nachricht senden —————
async function sendMessage() {
    const input = document.getElementById('chat-input')
    const text = input?.value?.trim()
    if (!text || state.streaming) return

    // User-Message hinzufügen
    state.messages.push({
        role: 'user',
        content: text,
        ts: Date.now()
    })

    // Input leeren
    input.value = ''
    input.dispatchEvent(new Event('input'))

    // Placeholder für Assistant-Antwort
    state.messages.push({
        role: 'assistant',
        content: '',
        ts: Date.now(),
        streaming: true
    })

    renderMessages()
    setStreaming(true)

    await streamCompletion()
}

// — SSE-Streaming —————
async function streamCompletion() {
    state.abortController = new AbortController()
    const startTs = Date.now()
    let tokenCount = 0

```

```

let firstTokenTs= 0

// Nachrichten für API zusammenstellen
const apiMessages = []
if (state.systemPrompt?.trim()) {
  apiMessages.push({ role: 'system', content: state.systemPrompt })
}
state.messages
  .filter(m => !m.streaming) // Aktueller Stream-Placeholder ausschließen
  .forEach(m => apiMessages.push({
    role: m.role,
    content: m.content
  }))

try {
  const resp = await fetch(
    `http://127.0.0.1:${state.port}/v1/chat/completions`,
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'hermes',
        messages: apiMessages,
        stream: true,
        temperature: state.temperature,
        max_tokens: state.maxTokens
      }),
      signal: state.abortController.signal
    }
  )

  if (!resp.ok) {
    throw new Error(`HTTP ${resp.status}: ${await resp.text()}`)
  }

  // — SSE Stream lesen —————
  const reader = resp.body.getReader()
  const decoder = new TextDecoder()
  let buffer = ''

  const currentMsg = state.messages[state.messages.length - 1]

  while (true) {
    const { done, value } = await reader.read()
    if (done) break

    buffer += decoder.decode(value, { stream: true })

    // SSE: durch \n\n getrennt
    const events = buffer.split('\n\n')
    buffer = events.pop() || '' // letzter unvollständiger Event zurück

    for (const event of events) {
      const lines = event.split('\n')
      for (const line of lines) {
        if (!line.startsWith('data: ')) continue
        const payload = line.slice(6).trim()

```

```

    if (payload === '[DONE]') {
      continue // Ende-Marker (OpenAI-Konvention)
    }

    try {
      const chunk = JSON.parse(payload)

      // — Standard OpenAI-Format —————
      const delta = chunk.choices?.[0]?.delta?.content
      if (delta) {
        if (!firstTokenTs) firstTokenTs = Date.now()
        tokenCount++
        currentMsg.content += delta
        updateCurrentMessage()
        updateStats(tokenCount, startTs, firstTokenTs)
      }

      // — llama.cpp non-OAI Format (Fallback) —————
      if (chunk.content && !delta) {
        if (!firstTokenTs) firstTokenTs = Date.now()
        tokenCount++
        currentMsg.content += chunk.content
        updateCurrentMessage()
        updateStats(tokenCount, startTs, firstTokenTs)
      }

      // — Stop-Signal —————
      if (chunk.choices?.[0]?.finish_reason || chunk.stop) {
        currentMsg.finishReason =
          chunk.choices?.[0]?.finish_reason || 'stop'
      }

    } catch (e) {
      console.warn('[Chat] SSE-Parse-Fehler:', e, payload)
    }
  }
}

// — Finalisieren —————
const elapsed = Date.now() - startTs
currentMsg.streaming = false
currentMsg.tokens = tokenCount
currentMsg.elapsed = elapsed
currentMsg.tokensPerSec =
  tokenCount > 0 ? (tokenCount / (elapsed / 1000)).toFixed(1) : 0

updateCurrentMessage()
renderMessages() // Final Render mit Metadaten

} catch (err) {
  const currentMsg = state.messages[state.messages.length - 1]
  if (err.name === 'AbortError') {
    currentMsg.content += '\n\n🛑 Generierung abgebrochen_'
  } else {
    currentMsg.content = `❌ **Fehler:** ${err.message}`
  }
}

```

```

        currentMsg.error = true
    }
    currentMsg.streaming = false
    renderMessages()
} finally {
    setStreaming(false)
    state.abortController = null
}
}

// — Streaming stoppen —————
function stopStreaming() {
    if (state.abortController) {
        state.abortController.abort()
    }
}

function setStreaming(streaming) {
    state.streaming = streaming
    const sendBtn = document.getElementById('chat-send-btn')
    const stopBtn = document.getElementById('chat-stop-btn')
    const input = document.getElementById('chat-input')

    if (sendBtn) {
        sendBtn.disabled = streaming || !input?.value?.trim()
        sendBtn.classList.toggle('hidden', streaming)
    }
    if (stopBtn) {
        stopBtn.classList.toggle('hidden', !streaming)
    }

    const status = document.getElementById('chat-status')
    if (status) {
        status.innerHTML = streaming
            ? '<span class="dot dot-on"></span> Generiere...'
            : '<span class="dot dot-on"></span> Server bereit'
    }
}

function updateStats(tokens, startTs, firstTokenTs) {
    const stats = document.getElementById('chat-stats')
    if (!stats) return
    const elapsed = (Date.now() - startTs) / 1000
    const tps = firstTokenTs
        ? (tokens / ((Date.now() - firstTokenTs) / 1000)).toFixed(1)
        : '0'
    const ttft = firstTokenTs
        ? ((firstTokenTs - startTs) / 1000).toFixed(2)
        : '?'
    stats.textContent =
        `${tokens} tok · ${tps} tok/s · TTFT ${ttft}s · ${elapsed.toFixed(1)}s`
}

// — Messages rendern —————
function renderMessages() {
    const container = document.getElementById('chat-messages')
    if (!container) return

```



```

if (state.messages.length === 0) {
  // Empty-State neu rendern
  renderChatPage(container.closest('.chat-layout').parentElement)
  return
}

container.innerHTML = state.messages.map((m, i) => `
  <div class="chat-msg chat-msg-${m.role} ${m.streaming ? 'streaming' : ''} ${m.error ? 'error' : ''}">
    <div class="msg-avatar">
      ${m.role === 'user' ? '👤' : '🤖'}
    </div>
    <div class="msg-body">
      <div class="msg-header">
        <span class="msg-role">${m.role === 'user' ? 'Du' : 'Hermes'}</span>
        <span class="msg-time">${formatTime(m.ts)}</span>
      </div>
      <div class="msg-content" id="msg-content-${i}">${renderMarkdown(m.content)}</div>
      ${m.tokens ? `
        <div class="msg-meta">
          ⚡ ${m.tokens} tok · ${m.tokensPerSec} tok/s · ${(m.elapsed/1000).toFixed(1)}s
          ${m.finishReason ? `· ${m.finishReason}` : ''}
        </div>
      ` : ''}
    </div>
  </div>
` ).join('')

container.scrollTop = container.scrollHeight
}

// — Nur Stream-Update (effizienter) —————
function updateCurrentMessage() {
  const idx = state.messages.length - 1
  const el = document.getElementById(`msg-content-${idx}`)
  if (el) {
    el.innerHTML = renderMarkdown(state.messages[idx].content) +
      '<span class="stream-cursor">█</span>'
    // Auto-Scroll
    const cont = document.getElementById('chat-messages')
    if (cont) cont.scrollTop = cont.scrollHeight
  } else {
    renderMessages()
  }
}

// — Minimaler Markdown-Renderer (kein CDN) —————
function renderMarkdown(text) {
  if (!text) return ''
  let html = escapeHtml(text)

  // Code-Blocks: ```lang\n...\n```
  html = html.replace(/```(\w*)\n([\s\S]*)```/g, (_, lang, code) =>
    `<pre class="code-block"><code class="lang-${lang}">${code}</code></pre>`
  )

  // Inline-Code: `code`

```

```

html = html.replace(/`([^`\\n]+)`/g, '<code class="inline-code">$1</code>')

// Bold: **text**
html = html.replace(/\\*\\*([^*\\n]+)\\*\\*/g, '<strong>$1</strong>')

// Italic: *text*
html = html.replace(/\\*([^*\\n]+)\\*/g, '<em>$1</em>')

// Links: [text](url)
html = html.replace(/\\[\\[\\(\\[\\]]+\\)\\]\\(\\([\\^\\]]+\\)\\)/g,
  '<a href="$2" data-url="$2" class="md-link">$1</a>'
)

// Headings: # Title
html = html.replace(/^### (.+)$/gm, '<h4>$1</h4>')
html = html.replace(/^## (.+)$/gm, '<h3>$1</h3>')
html = html.replace(/^# (.+)$/gm, '<h2>$1</h2>')

// Listen
html = html.replace(/^- (.+)$/gm, '<li>$1</li>')
html = html.replace(/(<li>.*<\\li>\\n?)+/g, m => `<ul>${m}</ul>`)

// Zeilenumbrüche
html = html.replace(/\\n/g, '<br>')

return html
}

const escapeHtml = s => String(s || '')
  .replace(/&/g, '&amp;')
  .replace(/</g, '&lt;')
  .replace(/>/g, '&gt;')

const formatTime = ts => new Date(ts).toLocaleTimeString('de-DE', {
  hour: '2-digit', minute: '2-digit', second: '2-digit'
})

// Externe Funktionen
function setPort(port) {
  state.port = port || 8080
  updateEndpoint()
  checkServerStatus()
}

return { renderChatPage, setPort }
})();

window.chatModule = chatModule

// Klick-Handler für Markdown-Links → in Browser öffnen
document.addEventListener('click', (e) => {
  if (e.target.classList?.contains('md-link')) {
    e.preventDefault()
    const url = e.target.dataset.url
    if (url) window.electronAPI.openExternalUrl(url)
  }
})

```

```
}  
})
```

## 4 renderer/pages/updater.js — Auto-Update-UI

{ } JavaScript



```
// =====  
//  UPDATER-UI – Banner + Dialog  
//  =====  
  
const updaterModule = (() => {  
  
  const state = {  
    status:    'idle', // idle | checking | available | downloading | downloaded | error | not-available  
    info:      null,  
    progress:  0,  
    error:     null,  
    bannerShown: false  
  }  
  
  // — Init: IPC-Events registrieren —————  
  function init() {  
    window.electronAPI.onUpdaterEvent(({ event, data }) => {  
      handleUpdaterEvent(event, data)  
    })  
  
    // Aktuelle Version im UI anzeigen  
    showVersionInfo()  
  }  
  
  async function showVersionInfo() {  
    const info = await window.electronAPI.updaterGetVersion()  
    const el = document.getElementById('app-version')  
    if (el) el.textContent = `v${info.version}`  
  }  
  
  // — Event-Handler —————  
  function handleUpdaterEvent(event, data) {  
    state.status = event  
  
    switch (event) {  
      case 'checking':  
        // Nur sichtbar wenn manuell ausgelöst  
        break  
  
      case 'available':  
        state.info = data  
        showUpdateBanner(data)  
        break  
    }  
  }  
})
```

```

    case 'not-available':
        // Optional Toast wenn manuell
        break

    case 'progress':
        state.progress = data.percent
        updateDownloadProgress(data)
        break

    case 'downloaded':
        state.info = data
        showInstallBanner(data)
        break

    case 'error':
        state.error = data.message
        showUpdateError(data.message)
        break
}
}

// — Update verfügbar Banner —————
function showUpdateBanner(info) {
    removeBanner()
    state.bannerShown = true

    const banner = document.createElement('div')
    banner.id = 'updater-banner'
    banner.className = 'updater-banner updater-available'
    banner.innerHTML = `
        <div class="ub-icon">📦</div>
        <div class="ub-content">
            <strong>Update verfügbar: v${info.version}</strong>
            <small>Aktuell installiert: v${info.currentVersion}</small>
        </div>
        <div class="ub-actions">
            <button class="btn btn-sm" onclick="updaterModule.showReleaseNotes()">
                📖 Details
            </button>
            <button class="btn btn-start btn-sm" onclick="updaterModule.startDownload()">
                ⬇️ Herunterladen
            </button>
            <button class="btn btn-sm" onclick="updaterModule.dismissBanner()">
                ✕
            </button>
        </div>
    `

    document.body.appendChild(banner)
}

// — Download-Progress aktualisieren —————
function updateDownloadProgress(data) {
    let banner = document.getElementById('updater-banner')

    if (!banner) {

```

```

    banner = document.createElement('div')
    banner.id = 'updater-banner'
    banner.className = 'updater-banner updater-downloading'
    document.body.appendChild(banner)
  } else {
    banner.className = 'updater-banner updater-downloading'
  }

  const mbDone = (data.transferred / 1024 / 1024).toFixed(1)
  const mbTot = (data.total / 1024 / 1024).toFixed(1)
  const mbSec = (data.bytesPerSec / 1024 / 1024).toFixed(1)

  banner.innerHTML = `
    <div class="ub-icon">⬇️</div>
    <div class="ub-content ub-content-full">
      <div class="ub-prog-header">
        <strong>Lade Update herunter... ${data.percent}%</strong>
        <small>${mbDone} / ${mbTot} MB · ${mbSec} MB/s</small>
      </div>
      <div class="ub-prog-bar">
        <div class="ub-prog-fill" style="width:${data.percent}%></div>
      </div>
    </div>
  `
}

// — Bereit zum Installieren —————
function showInstallBanner(info) {
  removeBanner()
  state.bannerShown = true

  const banner = document.createElement('div')
  banner.id = 'updater-banner'
  banner.className = 'updater-banner updater-ready'
  banner.innerHTML = `
    <div class="ub-icon">✅</div>
    <div class="ub-content">
      <strong>Update v${info.version} bereit</strong>
      <small>App wird neu gestartet um die Installation abzuschließen</small>
    </div>
    <div class="ub-actions">
      <button class="btn btn-start btn-sm" onclick="updaterModule.installNow()">
        🚀 Jetzt installieren
      </button>
      <button class="btn btn-sm" onclick="updaterModule.dismissBanner()">
        ⌚ Später
      </button>
    </div>
  `

  document.body.appendChild(banner)
}

function showUpdateError(message) {
  removeBanner()
  const banner = document.createElement('div')
  banner.id = 'updater-banner'
  banner.className = 'updater-banner updater-error'

```

```

    banner.innerHTML = `
      <div class="ub-icon">⚠️</div>
      <div class="ub-content">
        <strong>Update fehlgeschlagen</strong>
        <small>${message}</small>
      </div>
      <button class="btn btn-sm" onclick="updaterModule.dismissBanner()">X</button>
    `

    document.body.appendChild(banner)
    setTimeout(() => removeBanner(), 8000)
  }

  // — Aktionen —
  async function startDownload() {
    await window.electronAPI.updaterDownload()
  }

  async function installNow() {
    await window.electronAPI.updaterInstall()
  }

  function dismissBanner() {
    removeBanner()
  }

  function removeBanner() {
    const b = document.getElementById('updater-banner')
    if (b) b.remove()
    state.bannerShown = false
  }

  function showReleaseNotes() {
    if (!state.info) return
    alert(
      `Release Notes für v${state.info.version}: \n\n` +
      (state.info.releaseNotes || 'Keine Notes verfügbar')
    )
  }

  async function checkNow() {
    const result = await window.electronAPI.updaterCheck()
    if (!result.success) {
      alert(`Update-Check: ${result.error}`)
    }
  }

  return {
    init, checkNow, startDownload, installNow,
    dismissBanner, showReleaseNotes
  }
})()

window.updaterModule = updaterModule

// Auto-Init beim App-Start
updaterModule.init()

```

## 5 renderer/pages/downloads.js — Download-Manager

{ } JavaScript



```
// =====  
// DOWNLOAD-MANAGER – Echter Fortschrittsbalken für HF-Modelle  
// =====  
  
const downloadsModule = (() => {  
  
  const downloads = new Map() // downloadId → progress-state  
  
  function init() {  
    window.electronAPI.onDownloadEvent(({ downloadId, event, data }) => {  
      handleDownloadEvent(downloadId, event, data)  
    })  
  }  
  
  function handleDownloadEvent(downloadId, event, data) {  
    let dl = downloads.get(downloadId)  
  
    if (!dl) {  
      dl = { downloadId, status: 'pending', progress: 0 }  
      downloads.set(downloadId, dl)  
    }  
  
    Object.assign(dl, data, { status: event })  
  
    renderActiveDownloads()  
  
    if (event === 'completed') {  
      window.electronAPI.showNotification({  
        title: '✅ Download fertig',  
        body: `${dl.filename} || 'Datei' gespeichert.`  
      })  
    }  
  }  
  
  // — Neuer Download starten (von HF-Browser aufgerufen) —  
  async function startModelDownload(modelId, filename, fileUrl) {  
    // Zielordner wählen  
    const folderResult = await window.electronAPI.downloadPickFolder()  
    if (!folderResult.success) return null  
  
    const dlResult = await window.electronAPI.downloadStart({  
      downloadUrl: fileUrl,  
      filename,  
      targetDir: folderResult.path  
    })  
  }  
})
```

```

if (!dlResult.success) {
  alert(`Download-Fehler: ${dlResult.error}`)
  return null
}

// Tracking
downloads.set(dlResult.downloadId, {
  downloadId: dlResult.downloadId,
  filename,
  modelId,
  status: 'starting',
  progress: 0,
  targetPath: dlResult.targetPath
})

renderActiveDownloads()
showDownloadsBar()
return dlResult.downloadId
}

async function cancelDownload(downloadId) {
  if (!confirm('Download wirklich abbrechen?')) return
  await window.electronAPI.downloadCancel(downloadId)
}

// — Floating Bar mit aktiven Downloads —————
function renderActiveDownloads() {
  let bar = document.getElementById('downloads-bar')
  if (!bar) return

  const active = Array.from(downloads.values())
    .filter(d => d.status !== 'completed' && d.status !== 'cancelled')

  if (active.length === 0) {
    bar.style.display = 'none'
    return
  }

  bar.style.display = 'block'

  bar.innerHTML = `
    <div class="dlb-header">
      <strong>📁 Aktive Downloads (${active.length})</strong>
      <button class="btn btn-sm" onclick="downloadsModule.hideBar()"></button>
    </div>
    <div class="dlb-list">
      ${active.map(dl => renderDownloadItem(dl)).join('')}
    </div>
  `
}

function renderDownloadItem(dl) {
  const mbDone = ((dl.downloaded || 0) / 1024 / 1024).toFixed(1)
  const mbTot = ((dl.totalBytes || 0) / 1024 / 1024).toFixed(1)
  const mbSec = ((dl.speedBps || 0) / 1024 / 1024).toFixed(2)
  const eta = formatEta(dl.etaSeconds)

```





```
window.downloadsModule = downloadsModule
downloadsModule.init()
```

## 6 CSS für alle neuen Komponenten

{ } CSS



```
/* =====
CHAT INTERFACE
===== */

.chat-layout {
  display: flex;
  flex-direction: column;
  height: 100%;
  max-height: calc(100vh - 90px);
}

.chat-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding-bottom: 14px;
  border-bottom: 1px solid var(--border);
  margin-bottom: 14px;
}

.chat-title { display: flex; align-items: baseline; gap: 12px; }
.chat-title h1 { font-size: 1.2rem; }
.chat-endpoint {
  font-family: var(--font-mono);
  font-size: 0.78rem;
  color: var(--text-dim);
}

.chat-actions { display: flex; gap: 6px; }

.chat-options {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  margin-bottom: 14px;
}

.chat-options summary {
  padding: 10px 14px;
  cursor: pointer;
  font-size: 0.85rem;
  color: var(--text-dim);
}

.chat-options-grid { padding: 14px; }
.chat-options-grid textarea {
  width: 100%;
```

```

background: var(--surface2);
border: 1px solid var(--border);
color: var(--text);
padding: 8px;
border-radius: 6px;
font-family: inherit;
font-size: 0.85rem;
resize: vertical;
}

.chat-options-row { display: flex; gap: 14px; margin-top: 10px; }

.chat-messages {
  flex: 1;
  overflow-y: auto;
  padding: 16px;
  background: var(--bg);
  border-radius: 8px;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.chat-empty {
  text-align: center;
  padding: 40px 20px;
  color: var(--text-dim);
}

.chat-empty .ce-icon { font-size: 3rem; margin-bottom: 12px; }

.chat-suggestions {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  justify-content: center;
  margin-top: 20px;
}

.chat-suggestion {
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 8px 14px;
  border-radius: 20px;
  cursor: pointer;
  font-size: 0.82rem;
  transition: all 0.15s;
}

.chat-suggestion:hover {
  border-color: var(--accent);
  background: var(--surface2);
}

/* — Message — */

.chat-msg {
  display: flex;
  gap: 12px;
  animation: msgIn 0.15s ease;
}

@keyframes msgIn {

```

```
from { opacity: 0; transform: translateY(6px); }
to { opacity: 1; transform: translateY(0); }
}

.msg-avatar {
  width: 32px; height: 32px;
  flex-shrink: 0;
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1rem;
}

.chat-msg-assistant .msg-avatar {
  background: rgba(124,58,237,0.15);
  border-color: var(--accent);
}

.msg-body { flex: 1; min-width: 0; }
.msg-header {
  display: flex;
  align-items: baseline;
  gap: 10px;
  margin-bottom: 4px;
}

.msg-role { font-weight: 700; font-size: 0.85rem; }
.msg-time { font-size: 0.7rem; color: var(--text-dim); font-family: var(--font-mono); }

.msg-content {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 12px 14px;
  font-size: 0.88rem;
  line-height: 1.6;
  word-wrap: break-word;
}

.chat-msg-user .msg-content {
  background: rgba(124, 58, 237, 0.08);
  border-color: rgba(124, 58, 237, 0.3);
}

.chat-msg.error .msg-content {
  background: rgba(239, 68, 68, 0.08);
  border-color: var(--red);
}

.msg-content code.inline-code {
  background: var(--surface2);
  padding: 2px 6px;
  border-radius: 4px;
  font-size: 0.82rem;
  font-family: var(--font-mono);
}

.msg-content pre.code-block {
  background: var(--bg);
  border: 1px solid var(--border);
```

```

border-radius: 6px;
padding: 10px 14px;
overflow-x: auto;
margin: 10px 0;
font-size: 0.8rem;
}

.msg-content pre.code-block code { font-family: var(--font-mono); }
.msg-content a.md-link { color: var(--accent); cursor: pointer; }
.msg-content ul { padding-left: 20px; margin: 8px 0; }
.msg-content h2, .msg-content h3, .msg-content h4 { margin: 10px 0 6px; }

.msg-meta {
  font-size: 0.7rem;
  color: var(--text-dim);
  margin-top: 6px;
  font-family: var(--font-mono);
}

.stream-cursor {
  animation: blink 1s infinite;
  color: var(--accent);
  font-weight: 700;
}

@keyframes blink {
  0%, 50% { opacity: 1; }
  51%, 100% { opacity: 0; }
}

.chat-msg.streaming .msg-content {
  border-color: var(--accent);
  box-shadow: 0 0 0 1px rgba(124, 58, 237, 0.2);
}

/* — Input Bar ————— */
.chat-input-bar {
  display: flex;
  gap: 10px;
  padding: 12px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  margin-top: 14px;
}

.chat-input {
  flex: 1;
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 10px 12px;
  border-radius: 8px;
  font-family: inherit;
  font-size: 0.9rem;
  resize: none;
}

.chat-input:focus { outline: none; border-color: var(--accent); }
.chat-input-actions { display: flex; flex-direction: column; gap: 4px; justify-content: center; }

```

```

.chat-statusbar {
  display: flex;
  justify-content: space-between;
  padding: 6px 4px;
  font-size: 0.78rem;
  color: var(--text-dim);
  font-family: var(--font-mono);
}

/* =====
UPDATER BANNER
===== */

.updater-banner {
  position: fixed;
  top: 50px;
  right: 16px;
  z-index: 1500;
  display: flex;
  align-items: center;
  gap: 14px;
  background: var(--surface);
  border: 1px solid var(--accent);
  border-radius: 10px;
  padding: 14px 18px;
  min-width: 380px;
  max-width: 500px;
  box-shadow: 0 8px 30px rgba(0,0,0,0.4);
  animation: slideIn 0.3s ease;
}

@keyframes slideIn {
  from { transform: translateX(20px); opacity: 0; }
  to { transform: translateX(0); opacity: 1; }
}

.updater-banner.updater-error {
  border-color: var(--red);
}

.updater-banner.updater-ready {
  border-color: var(--green);
  box-shadow: 0 0 30px rgba(34, 197, 94, 0.3);
}

.ub-icon { font-size: 1.6rem; flex-shrink: 0; }
.ub-content { flex: 1; }
.ub-content strong { display: block; font-size: 0.9rem; margin-bottom: 2px; }
.ub-content small { color: var(--text-dim); font-size: 0.78rem; }

.ub-content-full { display: block; }
.ub-prog-header {
  display: flex;
  justify-content: space-between;
  margin-bottom: 6px;
}

.ub-prog-bar {
  height: 6px;

```

```

background: var(--surface2);
border-radius: 3px;
overflow: hidden;
}
.ub-prog-fill {
height: 100%;
background: var(--accent);
border-radius: 3px;
transition: width 0.3s;
}

.ub-actions { display: flex; gap: 6px; flex-shrink: 0; }

/* =====
   DOWNLOADS BAR (Floating)
   ===== */

.downloads-bar {
position: fixed;
bottom: 16px;
right: 16px;
z-index: 800;
background: var(--surface);
border: 1px solid var(--border);
border-radius: 10px;
width: 420px;
max-width: calc(100vw - 32px);
box-shadow: 0 8px 24px rgba(0,0,0,0.4);
overflow: hidden;
}
.dlb-header {
display: flex;
justify-content: space-between;
align-items: center;
padding: 10px 14px;
background: var(--surface2);
border-bottom: 1px solid var(--border);
}
.dlb-list {
max-height: 280px;
overflow-y: auto;
padding: 8px;
}

.dl-item {
display: flex;
align-items: center;
gap: 10px;
padding: 10px;
background: var(--bg);
border: 1px solid var(--border);
border-radius: 8px;
margin-bottom: 6px;
}
.dl-item:last-child { margin-bottom: 0; }
.dl-info { flex: 1; min-width: 0; }
.dl-name {

```

```

font-size: 0.82rem;
font-weight: 600;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
margin-bottom: 4px;
}
.dl-stats {
display: flex;
gap: 10px;
font-size: 0.7rem;
color: var(--text-dim);
font-family: var(--font-mono);
}
.dl-progress { width: 100px; flex-shrink: 0; }
.dl-prog-bar {
height: 6px;
background: var(--surface2);
border-radius: 3px;
overflow: hidden;
margin-bottom: 3px;
}
.dl-prog-fill {
height: 100%;
background: var(--accent);
border-radius: 3px;
transition: width 0.3s;
}
.dl-percent {
font-size: 0.7rem;
color: var(--text-dim);
font-family: var(--font-mono);
}

```

## 7 package.json — Vollständig mit electron-updater

{ } JSON



```

{
  "name": "hermes-control-center",
  "version": "2.5.0",
  "description": "Desktop-Launcher und Kontrollzentrum für lokale KI-Modelle",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "dev": "electron . --inspect",
    "build": "electron-builder --win --x64",
    "build:dir": "electron-builder --dir",
    "release": "electron-builder --publish always",
    "release:draft": "electron-builder --publish always --config.publish.releaseType=draft"
  }
}

```



```

},
"dependencies": {
  "electron-store": "^8.2.0",
  "electron-updater": "^6.3.9"
},
"devDependencies": {
  "electron": "^36.0.0",
  "electron-builder": "^24.13.3"
},
"build": {
  "appId": "com.hermes.controlcenter",
  "productName": "Hermes Control Center",
  "copyright": "Copyright © 2025",
  "asar": true,
  "win": {
    "target": [
      { "target": "nsis", "arch": ["x64"] }
    ],
    "icon": "assets/icon.ico",
    "requestedExecutionLevel": "asInvoker",
    "publisherName": "Hermes"
  },
  "nsis": {
    "oneClick": false,
    "perMachine": false,
    "allowToChangeInstallationDirectory": true,
    "createDesktopShortcut": true,
    "createStartMenuShortcut": true,
    "shortcutName": "Hermes Control Center",
    "uninstallDisplayName": "Hermes Control Center"
  },
  "publish": {
    "provider": "github",
    "owner": "DEIN_GITHUB_USERNAME",
    "repo": "hermes-control-center",
    "releaseType": "release"
  },
  "files": [
    "main.js",
    "preload.js",
    "renderer/**",
    "assets/**",
    "!node_modules/.cache",
    "!**/*.{md,log,txt}"
  ]
}
}

```

{ } Bash



```
# 1. Version in package.json hochsetzen
npm version patch # oder minor / major

# 2. GitHub Token setzen (einmalig)
$env:GH_TOKEN = "ghp_XXXXXXXXXXXX"

# 3. Bauen und auf GitHub Releases hochladen
npm run release

# 4. Auf GitHub: Release aus Draft veröffentlichen
#   → alle installierten Apps erhalten automatisch das Update
```

**Wichtig:**  *Der Auto-Update-Code darf nur in gepackten Apps laufen, nicht in der Entwicklung*  
— *app.isPackaged* Check ist deshalb im Code drin.

## Finale Feature-Matrix (Stand 2.5.0)

| Feature                        | Status | Datei        |
|--------------------------------|--------|--------------|
| Native Dateiauswahl            | ✓      | main.js      |
| llama-server spawn             | ✓      | main.js      |
| Live-Logs (stdout/stderr)      | ✓      | main.js      |
| Log-Suche, Filter, Export      | ✓      | app.js       |
| Health-Polling (5s)            | ✓      | main.js      |
| Tray-Icon + Kontextmenü        | ✓      | main.js      |
| Autostart ( wasOpenedAtLogin ) | ✓      | main.js      |
| Native Notifications           | ✓      | main.js      |
| Profil-System                  | ✓      | profiles.js  |
| HF Modell-Browser              | ✓      | hf-models.js |
| GPU-Monitor (nvidia-smi)       | ✓      | gpu.js       |
| API-Test (Chat Completions)    | ✓      | app.js       |
| Setup-Wizard                   | ✓      | app.js       |
| Settings-Page                  | ✓      | app.js       |
| Persistenz (electron-store)    | ✓      | main.js      |

| Feature                                                                                                   | Status | Datei                  |
|-----------------------------------------------------------------------------------------------------------|--------|------------------------|
| Einzelinstanz-Lock                                                                                        | ✓      | main.js                |
|  Auto-Updater (GitHub)    | ✓      | main.js + updater.js   |
|  Update-Banner UI         | ✓      | updater.js             |
|  In-App Download          | ✓      | main.js + downloads.js |
|  Download-Progress (live) | ✓      | downloads.js           |
|  Download-Cancel          | ✓      | main.js                |
|  Streaming-Chat (SSE)     | ✓      | chat.js                |
|  Abort-Controller         | ✓      | chat.js                |
|  Minimal-Markdown         | ✓      | chat.js                |
|  Token/Sek Stats          | ✓      | chat.js                |

## Bekannte llama.cpp-Eigenheiten (eingebaut)

 *Bekannter Bug (Sept 2025): SSE-Fehlermeldungen bei Context-Overflow sind nicht RFC-konform*

→ Eingebaut: Robuster try/catch um jedes `JSON.parse` + Fallback auf `chunk.content` Format

 *llama.cpp Stream-Terminator: ``data: {"content":"","stop":true,...}``*

→ Eingebaut: Beide Formate werden erkannt ( `finish_reason` und `stop:true` )

 *Neu in llama.cpp: Anthropic Messages API unter ``/v1/messages`` mit eigenem SSE-Format*

→ Optional erweiterbar: Aktuell nutzen wir OpenAI-kompatibles `/v1/chat/completions` , das ist universell

## Was du jetzt hast

Eine vollständige Produkt-DNA mit:

- Auto-Update aus GitHub Releases
- Modell-Downloads direkt aus der App mit echtem Fortschritt
- Vollwertigem Streaming-Chat zum schnellen Testen
- Allen vorherigen Features (Tray, Wizard, Profile, GPU, HF, Logs, Settings)

Empfehlung für Nächstes:

- E — Code-Signing-Setup für Windows (verhindert SmartScreen-Warnung)
- F — Erweiterte Markdown-Rendering (LaTeX, Mermaid-Diagramme)
- G — Multi-Conversation-System (Chat-Tabs mit Persistenz)
- H — RAG-Anbindung (lokale Dokumente einbinden)